

Three green apples are arranged in a cluster on a white background. One apple is in the foreground, slightly to the right, and two are behind it, one to the left and one to the right. The apples are bright green with some yellowing at the stems.

人工知能
IE229 - ARTIFICIAL INTELLIGENCE

第10回講義: PyTorch プログラミング (2)
Lecture 10: PyTorch Programming (2)

二宮 崇 (Takashi Ninomiya)
愛媛大学 (Ehime University)
ninomiya.takashi.mk@ehime-u.ac.jp

第10回はPyTorchプログラミングの続きです。

今回の講義内容

- **深層学習(ディープラーニング)演習**
 - Pythonのクラス
 - クラスを用いたPyTorchプログラミング
 - GPUを用いたMNISTの学習とテスト



2

今回はPythonのクラスについて勉強し、クラスを用いたPyTorchプログラミングとGPUを用いた画像認識演習を行います。

PYTHONのクラス



【復習】 Google Colaboratoryで PyTorchを動かす

- まず、import torchが問題なく動くか確認しよう

```
[1] import torch
```



```
[1] import torch
```



(参考)

Ctrl+Enter: セルの実行

(参考)

Alt+Enter: セルの実行+コードの追加

このようにエラーが特にでていなければok!

4

それではGoogle ColaboratoryでPyTorchを動かしてみましょう。ColaboratoryにはすでにPyTorchがインストールされているので、importを実行するだけで利用可能となります。まず、「import torch」を実行して何もエラーがでないことを確認しましょう。

クラス

● 独自のデータ型の定義

- メソッド（クラス用の関数）や属性（クラス用の変数）の定義が可能

クラスの定義

```
class Man:
    def __init__(self, name):
        self.name = name
        print('Initialized!')

    def hello(self):
        print('hello ' + self.name + '!')

    def goodbye(self):
        print('good bye ' + self.name + '!')

m = Man('Taro')
m.hello()
m.goodbye()
```

`__init__` : コンストラクタ
(初期化用メソッド)

`.`でメソッドや属性にアクセス
(クラス内では`self`.)

メソッドの第一引数には
`self`を明示的に書く

Man型オブジェクトの生成

実行結果

```
Initialized!
hello Taro!
good bye Taro!
```

5

Pythonのクラスを用いることで新しい型を作ることができます。クラスはユーザー定義の型を定義していて、データ構造と関数の両方をそのユーザー定義の型に提供します。

この図のクラスはManというクラスの例になります。class Man:で始まるスコープの範囲がManクラスの定義になります。この定義の後に、m = Man('Taro')を実行すると、まず、Man('Taro')によって、Man型のオブジェクトが作られ、それが変数mに格納されるようになります。リスト型や辞書型のオブジェクトを作るのと同じ様に、クラス名の後ろに括弧をつけることで、そのクラス名の新しい型のオブジェクトを作ることができます。

クラスはそういった意味で、どのようなオブジェクトを作るかということを表したテンプレートのようなものです。クラスによって生成されるオブジェクトにはそれぞれのオブジェクトが持つ固有のデータが格納されています。リスト型ならリストオブジェクトが保有するリスト構造が格納されています。ユーザー定義のクラスによる型のオブジェクトにもそれぞれオブジェクト固有のデータ構造をもたせることができます。このManクラスでは、nameという変数をそれぞれのオブジェクトが持っています。例えば、Man('Taro')はname変数に'Taro'という値を持っています。ここでn=Man('Jiro')とすると、nに格納されたオブジェクトにはname変数に'Jiro'という値が格納されて、mとnはそれぞれ別のname変数を持つことになります。このようなオブジェクト毎にもっている変数のことを属性と言います。

また、クラスには、クラス/オブジェクト固有の関数を定義する機能があります。このクラス/オブジェクト固有の関数のことをメソッドと言います。このManの例だと、__init__とhelloとgoodbyeがメソッドになります。__init__は特殊なメソッドで、コンストラクタと呼ばれます。Man('Taro')でMan型のオブジェクトを生成することができますが、このとき、Man型のオブジェクトをまず作って、そのあと、__init__(self, 'Taro')が呼び出されます。つまり、クラス名(引数の列)と書くことで、新しいオブ

ジェクトが生成されて__init__が呼び出される、ということになります。新しいオブジェクトを生成するための特殊なメソッドなので、コンストラクタと呼ばれています。

メソッドを呼び出すときは、オブジェクトに.(ドット)をつけてメソッド名を書くことになっています。つまり、m.hello()を実行すると、mという変数に格納されているMan型オブジェクトに対して、hello()というメソッドを呼び出します。メソッドはdef メソッド名(引数の列)という形式で定義されていますが、最初には必ずselfという引数を与えるようにします。m.hello()と実行したとき、そのm自身の属性やメソッドにアクセスすることになりますが、その際にselfを用いてアクセスします。つまり自分自身を指すオブジェクトがselfに入っているというわけです。helloの定義をみると、hello(self)となっていますが、このselfに自分自身が入っているというわけです。従って、self.nameにアクセスすることで、自分自身が持っているname変数の値にアクセスすることができる、というわけです。

属性はどこで定義しているのでしょうか？実は陽に属性を定義するのではなく、コンストラクタの中で、新しく属性を作る形で属性を定義しています。コンストラクタ__init__(self, name)をみると、まず、selfに新しく生成された自分自身のオブジェクトがはいっていて、nameという引数が渡されています。self.name = nameとすることでこのnameという引数の値をself.nameに代入しています。これで新しい属性を生成すると同時に、その初期値を定義していることになります。

クラスの継承

- 親クラス(基底クラス)に新しい機能を追加したクラス(派生クラス)を作りたい
 - 基底クラスの属性やメソッドを継承
 - 属性やメソッドの追加、上書き(オーバーライド)ができる

派生クラスの定義

```
class YoungMan(Man):
    def __init__(self, name):
        super(YoungMan, self).__init__(name)
        self.name = 'Mr. ' + self.name

    def hello(self):
        print('hi ' + self.name + '!')
```

class 派生クラス名(基底クラス名):

super(派生クラス名, self).メソッド名
で基底クラスのメソッドを呼び出せる

Helloメソッドを上書き

実行結果

```
m = YoungMan('Taro')
m.hello()
m.goodbye()
```

派生クラスのhelloが呼ばれる

基底クラスのgoodbyeが呼ばれる

```
Initialized!
hi Mr. Taro!
good bye Mr. Taro!
```

6

Pythonのクラスにはオブジェクト指向で採用されている継承機能もあります。

今すでに持っているクラスを拡張して新しいクラスを作りたいとなつたとき、その際に元のクラスを書き換えるとすでにあるソースコードが使えなくなったりいろいろと問題が起きます。そこで、Pythonには継承という機能がついていて、元のクラスを引き継いだ新しいクラスを簡単に定義することができます。拡張したい元のクラスのことは基底クラス(base class)と呼んで、拡張したクラスのことは派生クラス(derived class)と呼びます。継承機能により基底クラスの属性とメソッドをそのまま引き継いだ新しいクラスを定義することができます。

class 派生クラス名(基底クラス名):と書くことで基底クラスを継承した派生クラスを定義できます。この例の場合、Manが基底クラスで、YoungManが派生クラスになります。この宣言をするだけで、Manの__init__, hello, goodbyeが継承されます。(コンストラクタが継承されるので、属性も継承されることになります)

新しいメソッドをここに追加することもできますし、基底クラスがすでに持っているメソッドを上書きすることもできます。ここではコンストラクタとhelloを上書きしています。しかし、コンストラクタは属性も定義しているので、これを書き換えると、また属性を全て定義しなおす必要がでてきてしまいます。そこで、superという関数を用いると、基底クラスのメソッドを呼び出すことができます。super(派生クラス名, self).メソッド名で基底クラスのメソッドを呼び出せます。

ここではhelloは上書きしていますが、goodbyeは上書きしていません。そのため、helloは派生クラスの新しく定義したhelloが呼び出され、goodbyeは基底クラスのgoodbyeが呼び出されます。何らかの属性やメソッドを参照する際には、派生クラスから基底クラスの方(子孫から先祖の方)に向かって定義をみていって最初にみつかった定義を用います。

PYTORCHのテンソル型



Torch

- Pythonで高速に行列計算を行うためのライブラリ
 - 行列演算は単純な足し算掛け算を繰り返すがpythonはこういった計算が遅くて苦手
 - Pythonの便利さと高速計算を同時に実現する
- Torchのテンソル型(torch.tensor)には多次元配列を操作するための便利なメソッドが多数用意されている
- Torchをインポートして利用する

```
>>> import torch
```



8

TorchはPyTorchのライブラリ名です。Import torchで利用できます。

行列演算は単純な足し算掛け算を繰り返すがpythonはこういった計算が遅くて苦手です。例えば、単純なループだと、C言語の200～300倍ぐらい遅いです。

NumPyやTorchはPythonの便利さと高速計算を同時に実現します。

Torchのテンソル型は今まで勉強してきたように多次元配列を利用することができて、NumPyとほぼ同じ様に扱えます。

NumPyはCPUを使ったC言語での行列計算の高速化を目的としていて、PyTorchはGPUを使った行列計算の高速化を目的としています。また、前回勉強したように、PyTorchは勾配を計算することもできます。

torch.tensor：テンソル型の生成

● テンソル型の生成

- メソッドtensor()を使用する。引数はPythonのリスト。
- 多次元配列も生成可能。2次元配列は行列と捉えられる。
- メソッドsize()で配列の形状が得られる
- 属性dtypeに要素のデータ型が保存されている

2×3の行列生成

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$


```
>>> import torch
>>> x = torch.tensor([1.0, 2.0, 3.0])
>>> x
tensor([ 1.,  2.,  3. ])
>> type(x)
<class 'torch.Tensor'>
>>> A = torch.tensor([[1, 2, 3], [4, 5, 6]])
>>> A
tensor([[1, 2, 3],
        [4, 5, 6]])
>>> A.size()
torch.Size([2, 3])
>>> A.dtype
torch.int64
```

9

テンソル型についてももう少し詳しく勉強します。テンソル型を生成するにはtorch.tensorを用います。引数はPythonのリストです。二次元配列なら行列となりますし、二次元以上の多次元配列が扱えます。

テンソル型に対して、メソッドsize()を適用すると、配列の形状が得られます。テンソル型のdtype属性には要素のデータ型が保存されています。

torch.tensor：配列の演算 (1)

- 四則演算子による演算は**要素毎の演算**

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, B = \begin{pmatrix} -1 & 1 \\ 5 & 6 \end{pmatrix}$$

```
>>> x = torch.tensor([1., 2., 3.])
>>> y = torch.tensor([2., 4., 6.])
>>> x + y
tensor([3., 6., 9.])
>>> x - y
tensor([-1., -2., -3.])
>>> x * y
tensor([2., 8., 18.])
>>> x / y
tensor([0.5000, 0.5000, 0.5000])
```

要素毎の積

行列の積（ドット積）は
matmulメソッド

```
>>> A = torch.tensor([[1, 2], [3, 4]])
>>> B = torch.tensor([[ -1, 1], [5, 6]])
>>> A + B
tensor([[ 0, 3],
        [ 8, 10]])
>>> A * B
tensor([[ -1, 2],
        [15, 24]])
>>> torch.matmul(A, B)
tensor([[ 9, 13],
        [17, 27]])
```

$$\begin{pmatrix} 1 \times -1 & 2 \times 1 \\ 3 \times 5 & 4 \times 6 \end{pmatrix}$$

$$\begin{pmatrix} 1 \times -1 + 2 \times 5 & 1 \times 1 + 2 \times 6 \\ 3 \times -1 + 4 \times 5 & 3 \times 1 + 4 \times 6 \end{pmatrix}$$

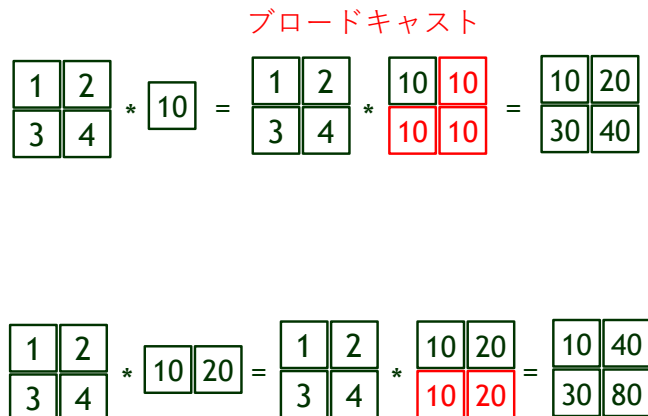
四則演算はNumPyと同様要素毎の演算になります。足し算、引き算は普通の演算と同じですが、掛け算はtorch.matmul(A, B)でABの行列積の計算ができることに注意しましょう。

10

torch.tensor：配列の演算 (2)

- 配列の形状が異なる場合は、形状を揃えて要素毎の演算がされる(ブロードキャスト)。

```
>>> A = torch.tensor([[1, 2], [3, 4]])
>>> A*10
tensor([[10, 20],
        [30, 40]])
>>> A+10
tensor([[11, 12],
        [13, 14]])
>>> B = torch.tensor([10, 20])
>>> A*B
tensor([[10, 40],
        [30, 80]])
```



11

NumPyやPyTorchでは配列の形状が異なる場合は、形状を揃えて要素毎の演算が行われます。この機能のことをブロードキャストと言います。

上の例では、2x2の行列と10の掛け算を計算していますが、要素積なのにサイズがありません。そのため、自動的に10の要素を2x2に拡張して、要素積の計算を行います。

下の例では2x2の行列と1x2の行列の要素積になりますが、これもサイズがないため、1x2の行列を2x2の行列に拡張して、それから要素積の計算を行います。

torch.tensor: 要素へのアクセス

- Pythonリストの要素へのアクセスと同様にインデックスによりアクセス可能
- 加えて、tensor型配列によるアクセスも可能
 - インデックスの配列により指定番目の要素だけ取得
 - ブーリアンの配列によりTrueに対応する要素を取得

tensor配列に対して不等号演算を行うと配列の各要素に対して不等号演算が行われた結果のブーリアンの配列となる

```
>>> X = torch.tensor([[10, 40], [15, 30], [1, 0]])
>>> X
tensor([[10, 40],
        [15, 30],
        [ 1,  0]])
>>> X[0] #0行目
tensor([10, 40])
>>> X[0][1] #(0,1)の要素
tensor(40)
>>> X = X.flatten()
>>> X
tensor([10, 40, 15, 30,  1,  0])
>>> X[torch.tensor([0,2,4])]
tensor([10, 15,  1])
>>> X>20
tensor([False,  True, False,  True, False, False])
>>> X[X>20]
tensor([40, 30])
```

Xを1次元配列に変換

12

要素へのアクセスはリストやNumPyでのアクセスと同じです。

X.flatten()は多次元配列を1次元配列に変換します。

X[torch.tensor([0,2,4])]で、テンソル型で表現されたインデックスを用いてアクセスすることもできます。

X>20を実行すると、Xの各要素に対して20より大きいかどうかの判定を行いブール型の値を返します。この場合、[False, True, False, True, False, False]というテンソル型が返ってきています。

ブール型の配列を用いることでTrueに対応する要素だけを抽出することもできます。組み合わせると、X[X>20]で、Xが20より大きな要素だけを集めたテンソル型を得ることができます。

PYTORCHによる深層学習



テンソル型の定義

- 計算グラフのノード(変数)を表す
- Pytorchではtorch.float32(実数), torch.float64(実数)の型を用いる

```
import torch

x = torch.zeros((10, 20), dtype=torch.float32, requires_grad=True)
print(x)
x = torch.ones((10, 20), dtype=torch.float32, requires_grad=True)
print(x)
x = torch.tensor([1,2,3], dtype=torch.float32, requires_grad=True)
print(x)
```



14

テンソル型の定義は前回勉強したとおりですが、今回はGPUを使うため、テンソル型の要素のデータ型に注意しましょう。

GPUが効率的に計算するためには、配列の中に整然と同じ大きさの要素が並んでいないといけません。(そうしないとメモリ空間の中で高速にアクセスできないし、また一番大きなサイズの型にあわせると、メモリの利用効率が非常に悪くなる)

そのため、テンソル型の中の要素は全て同じサイズの型になっています。テンソル型のオブジェクトを生成する際に型を指定することができて、dtype=torch.float32という感じで指定できます。

torch.int32

torch.int64

torch.float32

torch.float64

などが使えます。

torch.nn.functional

- パラメータ(求めたい重み変数)を含まない関数は torch.nn.functional パッケージに用意されています
 - 活性化関数
 - 損失関数など



15

PyTorchのライブラリについて説明していきます。まず、torch.nn.functionalというパッケージに、パラメータ(重み変数)を含まない関数が用意されています。ここに様々な活性化関数や損失関数などが用意されています。

torch.nn.functional

- 簡単な例

```
x1 = torch.tensor(1.5)
x2 = torch.tensor(0.75)
y = torch.nn.functional.relu(x1)
z = torch.nn.functional.mse_loss(x1, x2)
print("y: "+str(y))
print("z: "+str(z))
```

y: 1.5000
z: 0.5625



16

例えば、ReLUや、平均二乗誤差などが用意されています。

torch.nn

- パラメータ(求めたい重み変数)を含む関数はtorch.nnパッケージに用意されています。
 - 多くのNNモデルはtorch.nnの直下に用意されていて、活性化関数などの簡単な関数はtorch.nn.functionalの下に用意されている、ということになります。



17

パラメータ(重み変数)を含む関数は、torch.nnに用意されています。従って、多くのNNモデルはtorch.nnの直下に用意されていて、活性化関数などの簡単な関数はtorch.nn.functionalの下に用意されている、ということになります。

torch.nn.Parameter

- PyTorchでの重み変数(パラメータ)を生成するクラス

- テンソル型を生成する際にrequires_grad=Trueとするだけでは、pytorchの中ではパラメータ(保存と更新の対象となる変数)として認識されない
- nn.Parameter(x)とすることでxを中身とするパラメータを生成する(ただし、通常使うことはあまり無い。カスタムレイヤーを作るときに必要)
- 例: 2×3の重み行列

```
W = torch.nn.Parameter(torch.tensor([[1.,2.,3.],  
                                      [4.,5.,6.])))
```

↓

Parameter containing:

```
tensor([[1., 2., 3.],  
        [4., 5., 6.]], requires_grad=True)
```

18

torch.nn.Parameterは、PyTorchでの重み変数(パラメータ)を生成するクラスです。実は、テンソル型を生成する際にrequires_grad=Trueとするだけでは、pytorchの中ではパラメータ(保存と更新の対象となる変数)として認識されません。nn.Parameter(x)とすることでxを中身とするパラメータを生成することができます。ただし、通常このクラスを使うことはあまりありません。カスタムレイヤーを作るときには必要になります。

例として、2×3の重み行列を作ってみましょう。

```
W = torch.nn.Parameter(torch.tensor([[1,2,3], [4,5,6]]))
```

とすると、Wに含まれる値(変数)が全てパラメータとして扱われるようになります。

【復習】線形変換(全結合層) torch.nn.Linear

- torch.nn.Linear

- 線形変換(全結合層)のモジュール
 - $y = Wx + b$ を計算する
 - W (行列)と b (ベクトル)がパラメータ
- torch.nn.Linear(in_dim, out_dim)
 - in_dim次元のベクトルからout_dim次元のベクトルに射影する線形変換を計算する層を作る
- 例: torch.nn.Linear(3, 4)
 - 3次元ベクトルから4次元ベクトルに射影する線形変換



19

線形変換(全結合層のこと)を行うモジュールがtorch.nn.Linearです。このモジュールを用いることで、 $y = Wx + b$ を計算します。 W (行列)と b (ベクトル)がパラメータとなります。

torch.nn.Linear(in_dim, out_dim)と書くことで、in_dim次元のベクトルからout_dim次元のベクトルに射影する線形変換を計算する層を作ります。例えば、torch.nn.Linear(3, 4)と書くと、3次元ベクトルから4次元ベクトルに射影する線形変換を作れます。

torch.nn.Module

- **Moduleクラス**

- ネットワーク全体の構成(モデル)を与えるクラス
 - torch.nnやtorch.nn.functionalをパーツとして全体を構成し、そのネットワークが表現する合成関数を計算する
 - 合成関数に含まれるパラメータを学習する



20

続いて、Moduleクラスです。

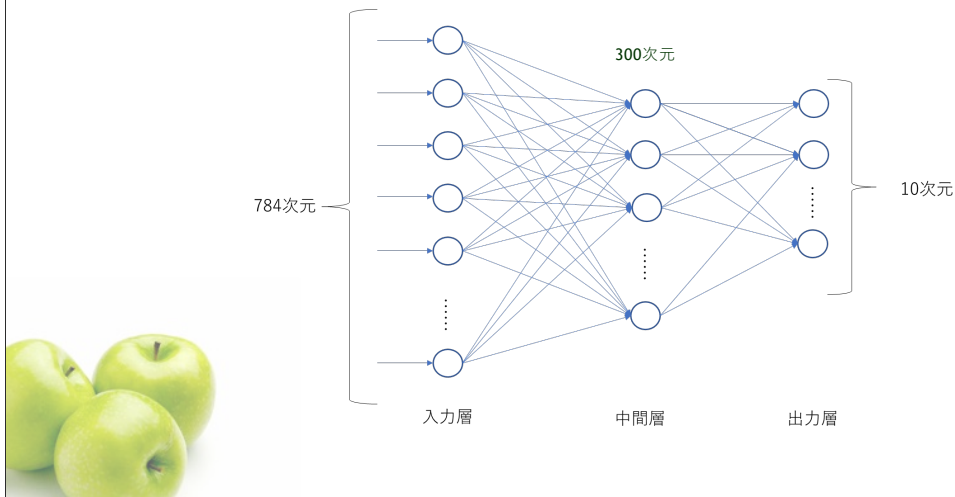
Moduleクラスはネットワーク全体の構成(モデル)を与えるクラスです。

torch.nnやtorch.nn.functionalをパーツとして全体を構成し、そのネットワークが表現する合成関数を計算します。optimizerと組み合わせることで、合成関数に含まれるパラメータを学習します。

今回はニューラルネットワークの計算を行う関数を直接書きましたが、PyTorchでは一般にこのModuleクラスを継承して、新しいニューラルネットワークを作ります。パラメータの更新、パラメータのセーブとロード、GPUの利用が容易にできるようになります。

Module

- 入力層(784次元)、中間層(300次元)、出力層(10次元)の3層のニューラルネットワークを作って白黒画像(28x28)の画像認識を行います



21

入力層(784次元)、中間層(300次元)、出力層(10次元)の3層のニューラルネットワークの例をみてみましょう。

Module

- 例: 入力層(784次元)、中間層(300次元)、出力層(10次元)の3層のニューラルネットワーク (活性化関数はReLU)

```
class MyNet(torch.nn.Module):  
    def __init__(self):  
        super(MyNet, self).__init__()  
        self.l1 = torch.nn.Linear(784, 300)  
        self.l2 = torch.nn.Linear(300, 10)  
    def forward(self, x):  
        h = torch.nn.functional.relu(self.l1(x))  
        y = torch.nn.functional.relu(self.l2(h))  
        return y
```

`__init__()`関数でネットワークの要素を定義

forward計算のときに
`forward()`が呼ばれる。
同時にネットワークを構築



22

`torch.nn.Module`を継承して、新しい自分用のニューラルネットワーククラスを作ります。ここでは`MyNet`という名前にしています。

`__init__`で基底クラスの初期化を行い、`l1`と`l2`に線形関数(全結合層)の関数を設定します。

`forward`は順伝播の計算を行うための関数です。入力`x`に対して、行うべき計算を定義します。ここでは、入力`x`を`l1`に適用し、`relu`をかけて`h`を出力し、`h`に対し`l2`を適用し、`relu`をかけて、最終出力の`y`を得ます。また、この`y`を出力として返します。

optim

- optimクラス

- 最適化ツール
- 様々な最適化アルゴリズムを選択して利用できる
- 最初に次の定義をして最適化クラスを選択、初期化

optimizerを選択し
パラメータをセット

```
model = MyNet()  
optimizer = torch.optim.SGD(model.parameters())
```

- パラメータ更新時

```
optimizer.zero_grad()  
loss = model(x)  
loss.backward()  
optimizer.step()
```

勾配の初期化

パラメータを更新

23

optimクラスは最適化ツールのクラスです。前回にもでてきました。今回はModuleクラスと連携します。

まず、Moduleクラスを継承したMyNetを使って、MyNet型のモデルmodelを作成します。そのモデルのパラメータをtorch.optim.SGDに渡して(torch.optim.SGD(model.parameters()))、optimizerを得ます。Modelの全パラメータ集合はmodel.parameters()で手に入ります。

パラメータ更新は同じように行うことができます。勾配の初期化はoptimizer.zero_grad()で実行できます。ニューラルネットワークモデル(model)を関数と見立てて、モデル(model)に入力(x)を関数適用の形(model(x))で与えます。関数適用の形で与えるとModuleクラスの中で、forward関数が呼ばれるようになっています。

パラメータ更新はoptimizer.step()で実行できます。

PYTORCHによるMNIST



データ MNIST

- **MNIST(前回用いたものと同じ)**

- 手書き文字の認識を行うタスク
- 画像に対して、どの数字が書かれているかを予測する。



- 訓練データは60,000データ
- テストデータは10,000データ
- 各データは画像とラベルのペア
- 各画像は28×28ピクセル
- ラベルは0～9までのいずれかの数字。画像に対する正解の数字。



25

データはMNISTと呼ばれるデータを用います。このデータは前回用いたものと同じです。

データの読み込み

- データ読み込みのためのプログラム(前回と同じ)

```
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F
import torchvision as tv
```

ライブラリの読み込み

```
train_dataset = tv.datasets.MNIST(root=".", train=True,
                                   transform=tv.transforms.ToTensor(),
                                   download=True)
test_dataset = tv.datasets.MNIST(root=".", train=False,
                                  transform=tv.transforms.ToTensor(),
                                  download=True)
train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
                                             batch_size=100,
                                             shuffle=True)
test_loader = torch.utils.data.DataLoader(dataset=test_dataset,
                                           batch_size=100,
                                           shuffle=False)
```

訓練データとテストデータの読み込み
(初めて実行するときはデータをネットからダウンロードする)

訓練データとテストデータのミニバッチ処理
・ミニバッチサイズ=100
・データの順番をシャッフル

26

まず、データを読み込むためのプログラムを書きましょう。これは前回と同じです。

グローバル変数の定義

- グローバル変数を定義しておきます

```
MODELNAME = "mnist.model"
EPOCH = 10
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(DEVICE)
```

GPUが利用可能ならDEVICE="cuda"
CPUを利用するのならDEVICE="cpu"



27

ここから新しくプログラムを書いていきます。

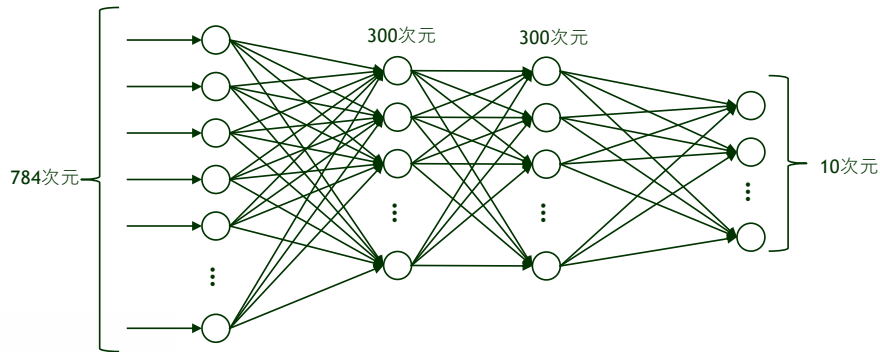
まず、グローバル変数を定義しておきます。モデルを保存するためのファイル名、エPOCH数、それから計算をどこで行うか指示するための変数DEVICEを用意しておきます。

DEVICEは、GPUが利用可能なら、DEVICE="cuda"となるように設定し、そうでないのならDEVICE="cpu"と設定するようにします。GPUが利用可能でもCPUを使いたい場合は、DEVICE="cpu"とすれば良いです。

モデルの定義

- **手書き文字の認識を行うモデル**

- 入力層(784次元)、中間層1(300次元)、中間層2(300次元)、出力層(10次元)の中間層2層のネットワーク



- 全結合層のみによるモデル
- `nn.Convolution2D`を使用し、入力データの形式を少し変更すれば、畳込みニューラルネットワーク (CNN) も可能

28

今回は、前回よりも複雑なニューラルネットワークを作ってみましょう。今回は3層のニューラルネットワークになります。中間層(300次元→300次元)が一つ増えています。

モデルの定義

● 手書き文字の認識を行うモデルを構築

- 入力層(784次元)、中間層1(300次元)、中間層2(300次元)、出力層(10次元)の中間層2層のネットワーク

```
class MNIST(torch.nn.Module):  
    def __init__(self):  
        super(MNIST, self).__init__()  
        self.l1 = torch.nn.Linear(784, 300)  
        self.l2 = torch.nn.Linear(300, 300)  
        self.l3 = torch.nn.Linear(300, 10)  
    def forward(self, x):  
        h = F.relu(self.l1(x))  
        h = F.relu(self.l2(h))  
        y = self.l3(h)  
        return y
```

__init__()関数でネットワークの要素を定義

forward計算のときにforward()が呼ばれる。同時にネットワークを構築

Softmax関数はここでは使わない。損失の計算のときに交差エントロピー損失と一緒に計算する

29

torch.nn.Moduleを継承して、MNISTという名前のニューラルネットワークを作ります。

コンストラクタでは、このニューラルネットワークで使用する線形関数l1, l2, l3を定義します。

forwardメソッドでは、入力xを受け取って、l1を適用した後、reluをかけて、その後、l2を適用し、またreluをかけています。最後は、l3を適用し、そのまま出力yとします。クラス分類なので最後にソフトマックス関数をかける必要があるような気がしますが、ソフトマックス関数は損失を計算するところで交差エントロピー損失と一緒に計算するためここではソフトマックス関数は使いません。これはソフトマックス関数と交差エントロピー損失を連結すると、その微分の計算が非常に簡単になるためです。

モデルの定義 `__init__`メソッド

- `__init__`メソッド

- 入力層の次元数: 768 (= ピクセル数)
- 第1中間層の次元数: 300
- 第2中間層の次元数: 300
- 出力次元数: 10 (= 多クラス分類のクラス数)
- ニューラルネットワークを定義する手順
 - 親クラスのコンストラクタを呼ぶ
 - `__init__()`: 以下に使用するネットワークを定義
- `torch.nn.Linear`: 全結合層
 - `torch.nn.Linear`の第一引数: 全結合層の入力次元数
 - `torch.nn.Linear`の第二引数: 全結合層の出力次元数
 - 全結合層以外にも多くのネットワークが使用できます (<https://pytorch.org/docs/stable/nn.html> 参照)。



30

`__init__`メソッドではまとめるとこのスライドに書かれていることを行っています。

モデルの定義 forwardメソッド

- **forwardメソッド**

- ネットワークの結合を定義
- 784次元のベクトル(28×28ピクセルの画像)を受け取る
- 10次元のベクトルを返す
- l1が784次元(入力層)を中間次元(300次元)に圧縮
- l2はl1の出力を受け取り、中間ベクトル(300次元)を生成
- l3は中間ベクトルを受け取り、10次元のベクトル(出力層)に変換
 - MNISTの分類が10クラス(0~9の文字)であるため、10次元のベクトルを出力
- l1, l2に対しては、活性化関数のrelu関数



31

forwardメソッドではまとめるとこのスライドに書かれていることを行っています。

モデルの訓練

```
def train():
    model = MNIST().to(DEVICE) #モデルを定義してDEVICEにのせる
    optimizer = torch.optim.Adam(model.parameters()) #最適化にAdamを使う
    for epoch in range(EPOCH): #EPOCH回最適化を行う
        loss = 0
        for images, labels in train_loader:
            images = images.view(-1, 28*28).to(DEVICE)
            labels = labels.to(DEVICE)
            optimizer.zero_grad() #勾配の初期化
            y = model(images) #forward計算
            batchloss = F.cross_entropy(y, labels) #損失の計算
            batchloss.backward() #逆伝搬の計算
            optimizer.step() #重み変数の更新
            loss = loss + batchloss.item()

        print("epoch", epoch, ": loss", loss)
    torch.save(model.state_dict(), MODELNAME)
```

データをミニバッチ
サイズに切り出す

(100×1×28×28)から
(100×784)に変形

ミニバッチ
内の処理

モデルをファイルに保存

32

続いて、モデルの訓練部分です。

まず、ニューラルネットワークモデルを作成します。

Model = MNIST().to(DEVICE)

によって、MNIST型のオブジェクトが作成されます。これがモデルとなります。
.to(DEVICE)というのがありますが、これはこの作成したオブジェクトをCPUが扱うメモリ上に置くか、GPUが扱うメモリに置くか指定するメソッドとなっています。
.to("cpu")とすると、CPUが扱うメインメモリ上に置かれることとなります。
.to("cuda")を指定すると、GPUが扱うメモリ上にこのモデルが置かれて、これらの計算はGPU上で行われるようになります。
.toで指定しないといけないオブジェクトは、このモデルと(学習用/テスト用)データです。

optimizer = torch.optim.Adam(model.parameters())とすることで、optimizerを用意します。

for epoch in range(EPOCH):によって、EPOCH回学習を行います。1エポックあたりの損失を計上するためにloss = 0としておきます。

続いて、ミニバッチの処理になります。for images, labels in train_loader:によってtrain_loaderからミニバッチのデータ(imagesとlabels)が渡されます。imagesは前回説明したように、viewによって、(100×1×28×28)のテンソルから、(100×784)のテンソルに変形します。imagesとlabelsをGPU上に置く必要がありますので、.to(DEVICE)を追加しておきます。

その後、optimizer.zero_gradで勾配の初期化を行い、y=model(images)を実行することで、imagesをまとめてmodelに渡して計算グラフを作りながら順伝播の計算を行います(MNISTクラスのforwardメソッドの計算を行います)。

ミニバッチでの損失は交差エントロピーで計算します。これは、batchloss = F.cross_entropy(y, labels)で計算できます。ただし、このF.cross_entropyは実際にはソフトマックス関数と交差エントロピー損失をあわせたものになっていることに注

意してください。

その後、損失に対する逆伝播の計算を`batchloss.backward()`で計算します。

続いて、`optimizer.step()`でパラメータの更新を行い、`loss = loss + batchloss.item()`でミニバッチの損失を`loss`に追加します。

全データの計算を終わったところで、エポック数と損失を表示します。

EPOCH回上記の計算を繰り返したあと、最後に学習結果であるモデルを保存します。`torch.save(model.state_dict(), MODELNAME)`でMODELNAMEのファイルにモデルの状態が保存されます。

テスト

```
def test():
    total = len(test_loader.dataset)
    correct = 0
    model = MNIST().to(DEVICE)
    model.load_state_dict(torch.load(MODELNAME))
    model.eval()
    for images, labels in test_loader:
        images = images.view(-1, 28*28).to(DEVICE)
        labels = labels.to(DEVICE)
        y = model(images) #forward計算
        pred_labels = y.max(dim=1)[1]
        correct = correct + (pred_labels == labels).sum()
    print("correct:", correct.item())
    print("total:", total)
    print("accuracy:", (correct.item() / float(total)))
```

ファイルに保存したモデルをロード

テストデータに対してループ

最大値の次元IDを得る

正解率を計算

33

最後にテストセットに対する評価プログラムです。

totalとcorrectは前回と同じで、totalは全データ数、correctはこれから計算する正解数を表します。

学習のときと同じ様にモデルをまず構築します。

model.load_state_dict(torch.load(MODELNAME))によって、学習時に保存したモデルをロードすることができます。

ここで、model.eval()というのが入っていますが、これはモデル内の構造をテスト用に設定するためのメソッドで、学習時には必要だったけど、テスト時には不要な計算を省いて効率化してくれます。

imagesを.to(DEVICE)でGPU上に転送して、出力の予測を行います。得られた出力をlabelsと比較することで正解率を計算します。

訓練・テスト

```
train()  
#test()
```

訓練・テストのうちやりたくないこと
に対してコメントアウトする

Pythonでは#がコメントを表す記号

- train()を実行して訓練してみよう
- test()を実行して精度を測ってみよう
- 各データに対し、正解ラベルと予測ラベルをそれぞれ表示させてみよう
- <http://yann.lecun.com/exdb/mnist/> の結果と比較してみよう



34

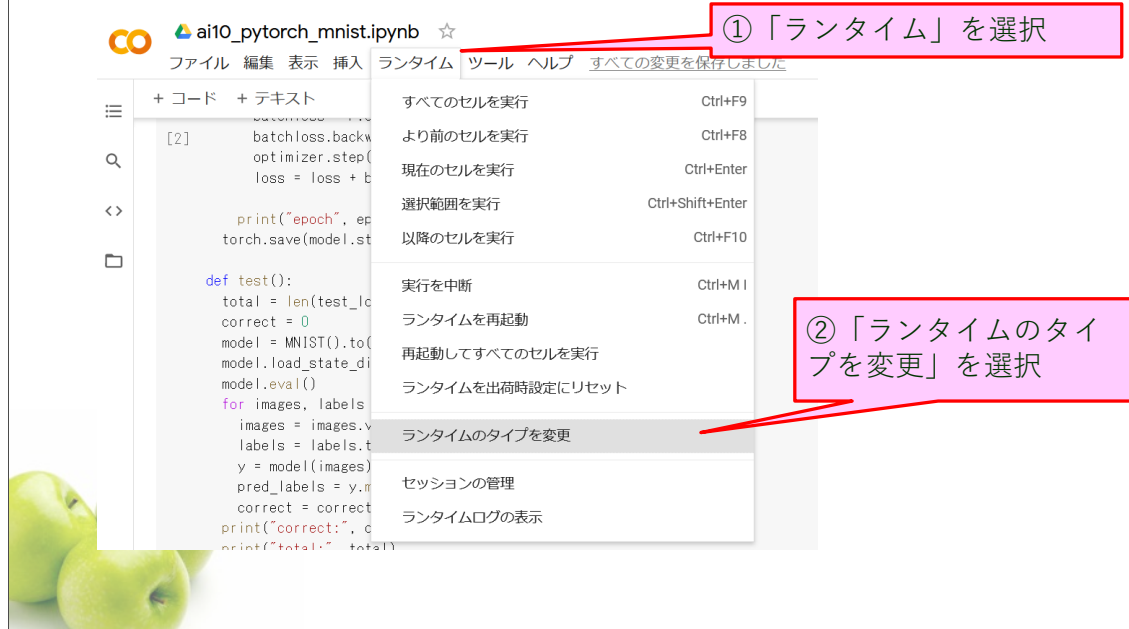
train()を実行すると学習が行われ、test()を実行するとテストが行われます。最初は両方実行すると良いのですが、一度学習してしまえばまた学習する必要は無いので、testだけ実行すれば良いようになります。学習するかテストを行うかは、このtrain()とtest()をコメントアウトすることで選ぶことができます。コメントアウトは#をつけることでできます。

各データに対し、正解ラベルと予測ラベルがどうなっているかそれぞれ表示させてみましょう。

<http://yann.lecun.com/exdb/mnist/> に様々なモデルでの結果が載っているので、これらの結果と比較してみましょう。

GPUを使ってみよう

- Google ColaboratoryではGPUを利用することができます



The screenshot shows the Google Colaboratory interface for a notebook named 'ai10_pytorch_mnist.ipynb'. The 'Runtime' menu is open, displaying various execution options. Two callouts highlight specific actions:

- ① 「ランタイム」を選択 (Select 'Runtime') - points to the 'ランタイム' menu item.
- ② 「ランタイムのタイプを変更」を選択 (Select 'Change runtime type') - points to the 'ランタイムのタイプを変更' option within the 'Runtime' menu.

The code editor shows a PyTorch MNIST training script. The 'Runtime' menu options include: 全てのセルを実行 (Ctrl+F9), より前のセルを実行 (Ctrl+F8), 現在のセルを実行 (Ctrl+Enter), 選択範囲を実行 (Ctrl+Shift+Enter), 以降のセルを実行 (Ctrl+F10), 実行を中断 (Ctrl+M I), ランタイムを再起動 (Ctrl+M .), 再起動してすべてのセルを実行, ランタイムを出荷時設定にリセット, ランタイムのタイプを変更 (highlighted), セッションの管理, and ランタイムログの表示.

35

最後にGPUを使って学習を行っています。おそらく今までは特に設定していなければCPUを使っていたと思います。Google ColaboratoryではGPUが使えますので、是非使って体験してみましょう。「ランタイム」を選択して、「ランタイムのタイプを変更」を選択します。

GPUを使ってみよう

- GPUの設定ができればMNISTの学習を行ってみよう



ランタイムのタイプを変更

ランタイムのタイプ

Python 3

ハードウェア アクセラレータ ?

☐ CPU ☒ T4 GPU ☐ A100 GPU ☐ V100 GPU

☐ TPU

プレミアム GPU を利用するには [コンピューティングユニットを追加購](#)

キャンセル 保存

ハードウェアアクセラレータの選択ボックスで、「T4 GPU」を選択し、保存をクリックします。これで準備okです。

先程の学習をこの設定でやって、CPUの場合の学習時間とGPUの場合での学習時間を比較してみましょう。

まとめ

- Pythonのクラスについて学んだ
- テンソル型について勉強した
- クラスを用いたPyTorchの使い方を学んだ
- GPUを用いた深層学習を行った



37

今回のまとめです。